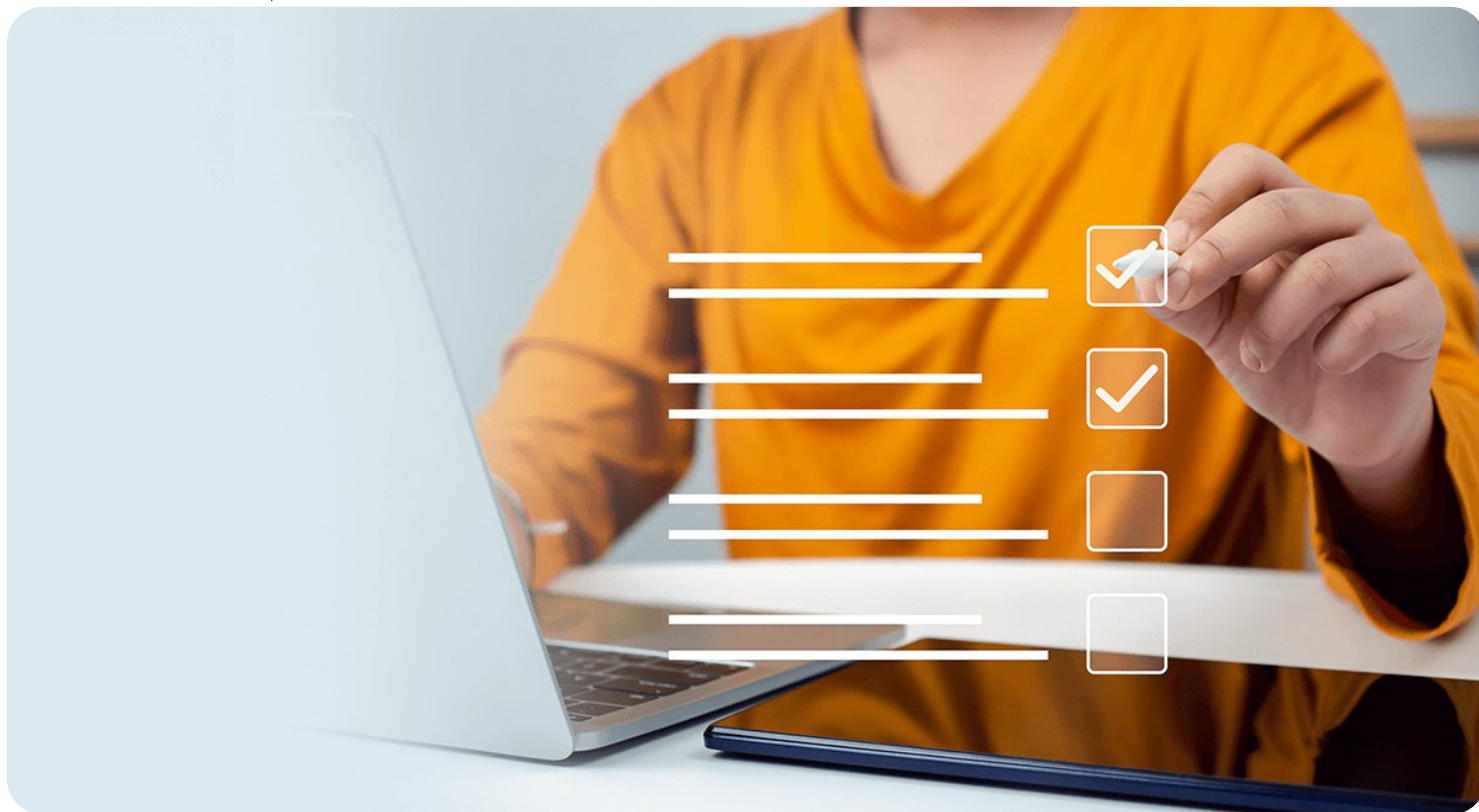




Semana 4

Desarrollo Backend III

(PBY2203)

Nombres estudiantes:	Egor Llanapichun Natalia Alvarado
Asignatura: Desarrollo Backend III	Carrera: Analista Programador Computacional
Profesor: Alonso Castillo	Fecha: 07-09-2026

Índice

1. Introducción
2. Análisis de la solución BFF
3. Arquitectura implementada
4. Implementación del BFF Web
5. Implementación del BFF Mobile
6. Implementación del BFF ATM
7. Seguridad y autenticación
8. Pruebas y resultados
9. Conclusión

1. Introducción

El presente informe describe la implementación del patrón arquitectónico Backend for Frontend (BFF) para el sistema Banco XYZ. La solución busca adaptar el acceso a los servicios bancarios según las necesidades de los clientes Web, Mobile y ATM.

Para esto se implementó un backend central encargado de exponer los datos procesados y tres BFF independientes, uno para cada canal. Los BFF consumen los servicios del backend mediante HTTP y adaptan las respuestas antes de entregarlas al cliente, evitando el acceso directo a la base de datos.

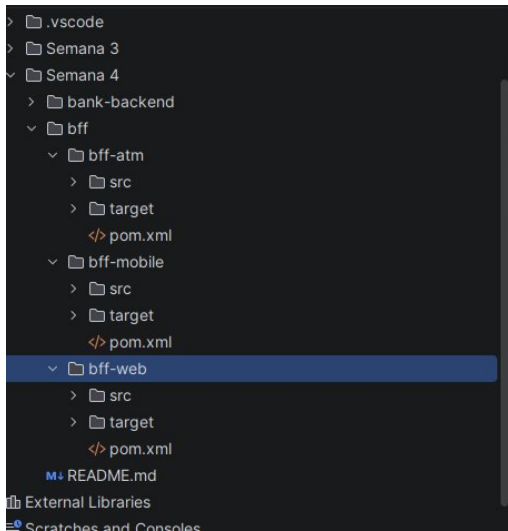
El objetivo de la actividad es aplicar el patrón BFF para separar las necesidades de cada canal, optimizar la información entregada y establecer mecanismos de autenticación diferenciados.

2. Análisis de la solución BFF

La utilización del patrón Backend for Frontend permite adaptar los servicios del sistema según las características de cada cliente. En lugar de entregar la misma respuesta a todos los consumidores, cada BFF controla la información que necesita su respectivo canal.

Para el canal Web se definieron respuestas con mayor cantidad de información, considerando interfaces que pueden requerir datos más completos. Para Mobile se implementaron respuestas reducidas, con el objetivo de disminuir el consumo de datos y entregar solamente la información necesaria. Para ATM se priorizaron operaciones simples y críticas, como la consulta de saldo y el retiro de dinero.

La solución también permite mantener una separación entre los clientes, los BFF y el backend principal. Los BFF se comunican con el backend mediante HTTP utilizando WebClient y no realizan conexiones directas con PostgreSQL.



La estructura muestra la separación del backend principal y los tres BFF independientes, cada uno destinado a un canal específico.

3. Arquitectura implementada

La solución implementada se compone de un backend central y tres Backend for Frontend independientes. El backend principal se encuentra conectado a PostgreSQL y expone los servicios utilizados por los distintos BFF.

El BFF Web se ejecuta en el puerto 8081, el BFF Mobile en el puerto 8082 y el BFF ATM en el puerto 8083. Cada BFF utiliza WebClient para comunicarse mediante HTTP con el backend principal, que se ejecuta en el puerto 8080.

Esta separación permite que cada canal disponga de endpoints y respuestas adaptadas a sus necesidades, manteniendo además el acceso a la base de datos encapsulado dentro del backend.

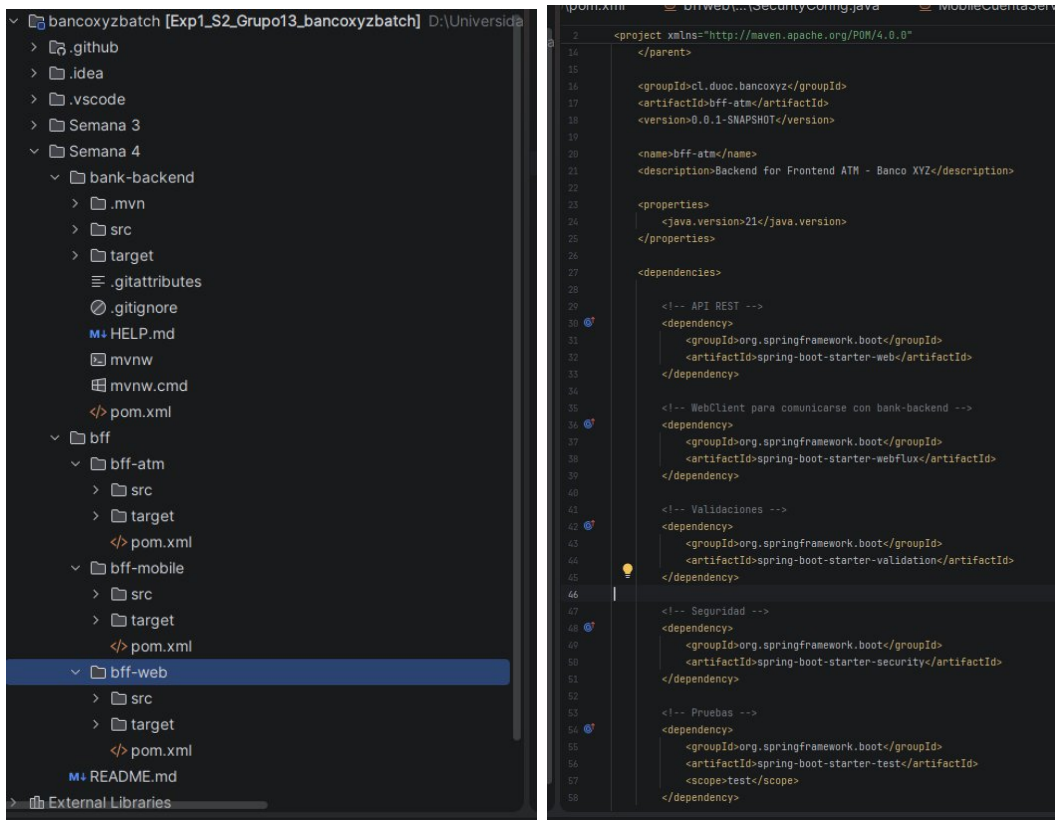


Figura 2. Arquitectura de aplicaciones y separación de componentes del proyecto.

Descripción:

Se observa la separación del backend principal y los tres BFF implementados para los canales Web, Mobile y ATM.

Figura 3. Dependencias principales utilizadas en el backend central.

El archivo `pom.xml` muestra las principales tecnologías utilizadas para construir el backend, incluyendo Spring Boot, acceso mediante JPA y el controlador de PostgreSQL.

4. Implementación del BFF Web

El BFF Web fue implementado como un servicio independiente que se ejecuta en el puerto **8081**. Su función es consumir los servicios del backend central y entregar información más completa, adecuada para interfaces web que requieren mayor cantidad de datos.

La comunicación con el backend se realiza mediante **WebClient**, utilizando los endpoints disponibles en el puerto 8080. Además, se implementó un endpoint de detalle que combina información de la cuenta y sus estados de cuenta.

Endpoints principales:

- GET /api/web/cuentas
- GET /api/web/cuentas/{id}
- GET /api/web/cuentas/{id}/detalle

```
//
package cl.duoc.bancoxyz.bffweb.service;

import javax.ws.rs.*;

@Service
public class WebCuentaService {
    private final WebClient webClient;

    public WebCuentaService(WebClient webClient) {
        this.webClient = webClient;
    }

    public Object obtenerCuentas() {
        return this.webClient.get().uri("/api/cuentas", new Object[0]).retrieve().bodyToMono(Object.class).block();
    }

    public Object obtenerCuenta(Long id) {
        return this.webClient.get().uri("/api/cuentas/{id}", new Object[]{id}).retrieve().bodyToMono(Object.class).block();
    }

    public WebCuentaDetalle obtenerDetalleCuenta(Long id) {
        Object cuenta = this.webClient.get().uri("/api/cuentas/{id}", new Object[]{id}).retrieve().bodyToMono(Object.class).block();
        Object estadosCuenta = this.webClient.get().uri("/api/estados-cuenta/cuenta/{cuentaId}", new Object[]{id}).retrieve().bodyToMono(Object.class).block();
        return new WebCuentaDetalle(cuenta, estadosCuenta);
    }
}
```

Figura 4. Implementación del servicio del BFF Web mediante WebClient.

El servicio utiliza WebClient para consumir el backend central y combinar información de cuentas y estados de cuenta, entregando una respuesta adaptada a las necesidades del canal Web.

5. Implementación del BFF Mobile

El BFF Mobile fue implementado como un servicio independiente que se ejecuta en el puerto 8082. Su objetivo es entregar respuestas reducidas y enfocadas en la información esencial, disminuyendo la cantidad de datos enviados al cliente móvil.

Para esto, el BFF consume el backend central mediante WebClient y transforma la información de las cuentas utilizando el DTO MobileCuentaResumen, que incluye únicamente los datos necesarios para la aplicación móvil.

Endpoints principales:

- GET /api/mobile/cuentas
- GET /api/mobile/cuentas/{id}/resumen

```
MobileCuentaService.java
13 private final WebClient webClient;
14
15 > public MobileCuentaService(WebClient webClient) { this.webClient = webClient; }
16
17 < public List<MobileCuentaResumen> obtenerCuentas() {
18
19     List<Map<String, Object>> cuentas = webClient
20         .get()
21         .uri("/api/cuentas")
22         .retrieve()
23         .bodyToMono(List.class)
24         .block();
25
26     return cuentas.stream()
27         .map(cuenta -> new MobileCuentaResumen(
28             ((Number) cuenta.get("cuentaId")).longValue(),
29             (String) cuenta.get("nombre"),
30             (String) cuenta.get("tipo"),
31             (String) cuenta.get("estado"),
32             cuenta.get("saldoFinal")
33         ))
34         .toList();
35 }
36
37 < public MobileCuentaResumen obtenerResumen(Long id) {
38
39     Map<String, Object> cuenta = webClient
40         .get()
41         .uri("/api/cuentas/{id}", id)
42         .retrieve()
43         .bodyToMono(Map.class)
44         .block();
45
46     return new MobileCuentaResumen(
47         ((Number) cuenta.get("cuentaId")).longValue(),
48         (String) cuenta.get("nombre"),
49         (String) cuenta.get("tipo"),
50         (String) cuenta.get("estado"),
51         cuenta.get("saldoFinal")
52     );
53 }
```

Figura 5. Implementación del servicio del BFF Mobile y adaptación de respuestas.

El servicio transforma la información recibida desde el backend y entrega únicamente los datos esenciales de la cuenta, optimizando la respuesta para un cliente móvil.

6. Implementación del BFF ATM

El BFF ATM fue implementado como un servicio independiente que se ejecuta en el puerto 8083. Su objetivo es proporcionar operaciones simples y eficientes para un cajero automático, priorizando la rapidez y seguridad de las operaciones críticas.

El BFF permite consultar el saldo disponible de una cuenta y realizar retiros, comunicándose con el backend central mediante WebClient.

Endpoints principales:

- GET /api/atm/cuentas/{id}/saldo
- POST /api/atm/cuentas/{id}/retiro

Además, el retiro valida que el monto sea válido y que exista saldo suficiente antes de modificar el saldo de la cuenta.

```
@Service
public class AtmCuentaService {

    private final WebClient webClient;

    public AtmCuentaService(WebClient webClient) {
        this.webClient = webClient;
    }

    public AtmCuentaSaldo consultarSaldo(Long id) {

        Map<String, Object> cuenta = webClient
            .get()
            .uri("/api/cuentas/{id}", id)
            .retrieve()
            .bodyToMono(Map.class)
            .block();

        return new AtmCuentaSaldo(
            ((Number) cuenta.get("cuentaId")).longValue(),
            (String) cuenta.get("nombre"),
            cuenta.get("saldoFinal")
        );
    }

    public Object retirar(Long id, BigDecimal monto) {

        AtmRetiroRequest request = new AtmRetiroRequest(monto);

        return webClient
            .post()
            .uri("/api/cuentas/{id}/retiro", id)
            .bodyValue(request)
            .retrieve()
            .bodyToMono(Object.class)
            .block();
    }
}
```

El servicio adapta las operaciones de consulta de saldo y retiro para el canal ATM, manteniendo las operaciones simples y enfocadas en las funciones esenciales del cajero.

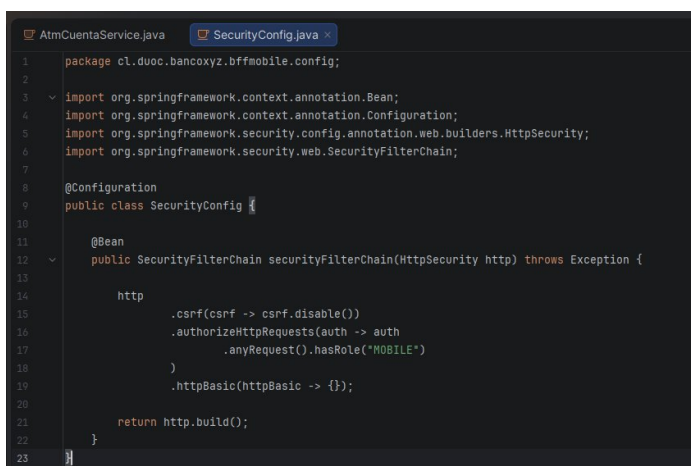
7. Seguridad y autenticación

Se implementó autenticación mediante **HTTP Basic** en cada BFF, utilizando credenciales y roles diferenciados según el canal de acceso. De esta forma, cada servicio puede controlar quién tiene autorización para utilizar sus endpoints.

Las credenciales configuradas son:

- **Web:** usuario web, rol WEB.
- **Mobile:** usuario mobile, rol MOBILE.
- **ATM:** usuario atm, rol ATM.

Además, se deshabilitó CSRF debido a que los BFF funcionan como APIs REST y se protegieron sus endpoints mediante Spring Security.



```
1 package cl.duoc.bancoxyz.bffmobile.config;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
6 import org.springframework.security.web.SecurityFilterChain;
7
8 @Configuration
9 public class SecurityConfig {
10
11     @Bean
12     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
13
14         http
15             .csrf(csrf -> csrf.disable())
16             .authorizeHttpRequests(auth -> auth
17                 .anyRequest().hasRole("MOBILE")
18             )
19             .httpBasic(httpBasic -> {});
20
21         return http.build();
22     }
23 }
```

Figura 6. Configuración de autenticación y autorización mediante Spring Security.

La configuración de seguridad protege los endpoints del BFF mediante autenticación HTTP Basic y restringe el acceso según el rol correspondiente a cada canal.

8. Pruebas y resultado

Se realizaron pruebas utilizando **Postman** para verificar el funcionamiento de los BFF y su comunicación con el backend central. Se comprobó que cada canal puede acceder a sus endpoints correspondientes y recibir la información adaptada. Las principales pruebas realizadas fueron:

- Consulta de cuentas mediante el **BFF Web**.
- Consulta de resumen de cuenta mediante el **BFF Mobile**.
- Consulta de saldo mediante el **BFF ATM**.
- Ejecución de un retiro mediante el **BFF ATM**.
- Verificación de autenticación mediante credenciales válidas.

Los resultados obtenidos fueron satisfactorios, obteniendo respuestas **HTTP 200 OK** en las operaciones correctamente autenticadas.

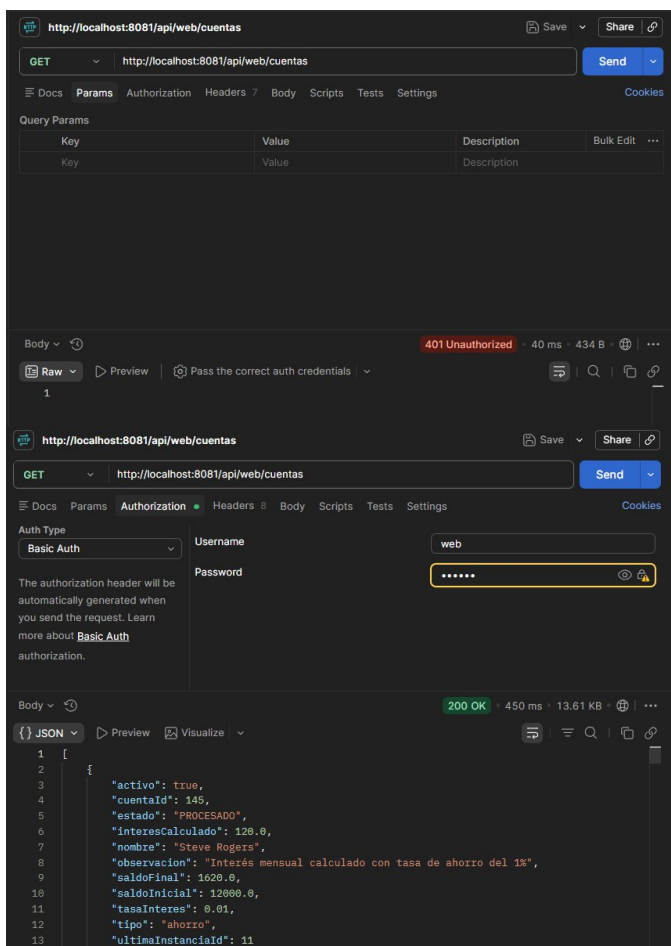


Figura 8. Prueba de consulta de cuentas mediante BFF Web.

Se verifica la obtención de información desde el BFF Web mediante una solicitud autenticada.

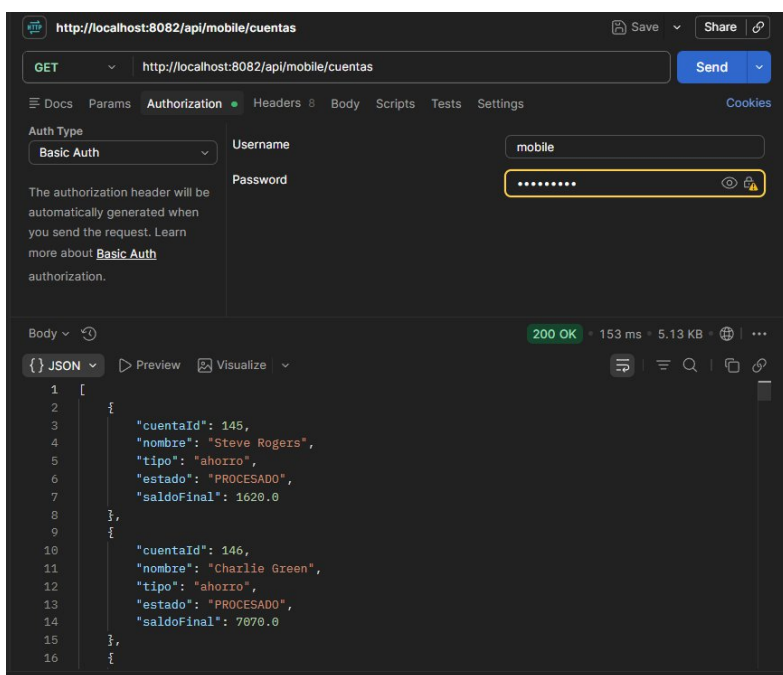
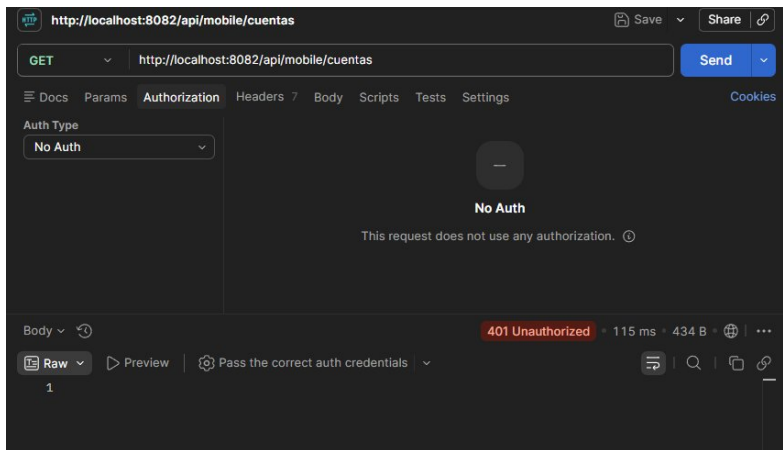


Figura 9. Prueba de respuesta optimizada mediante BFF Mobile.

Se observa que el BFF Mobile entrega únicamente la información esencial de la cuenta.

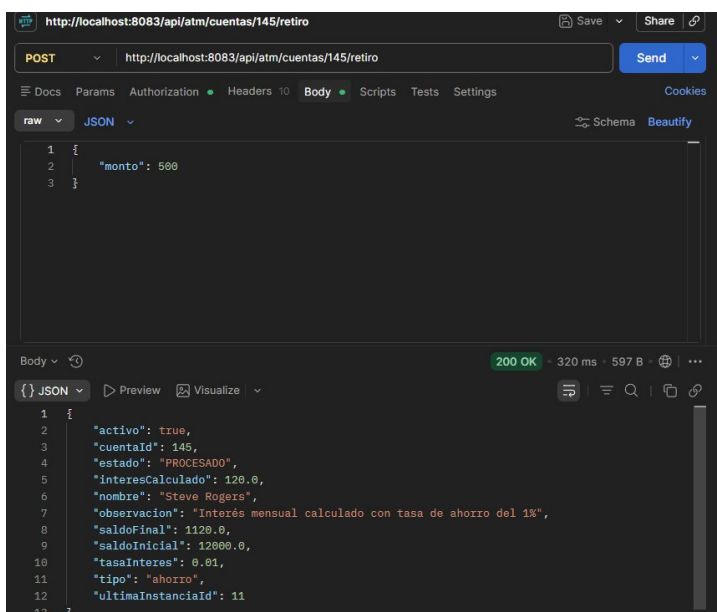
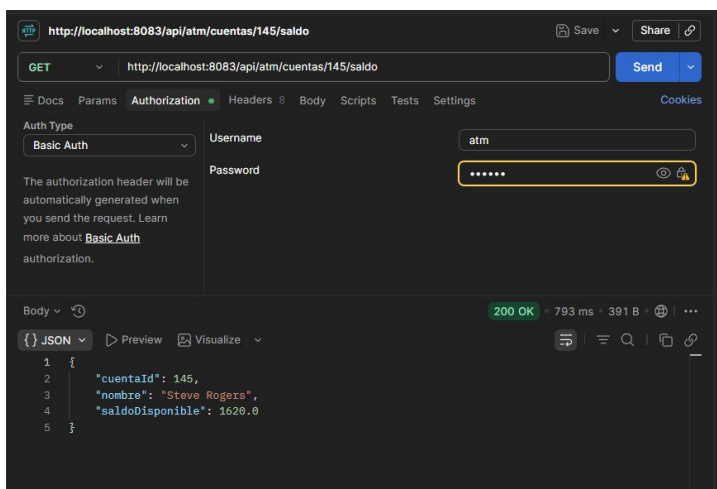
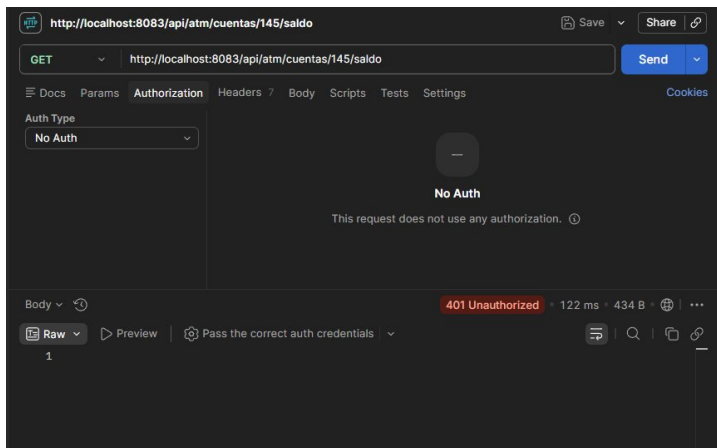


Figura 9. Prueba de consulta de saldo y retiro mediante BFF ATM.

Se comprueba el funcionamiento de las operaciones críticas del canal ATM, incluyendo la consulta de saldo y el retiro.

9. Conclusión

La implementación del patrón **Backend for Frontend (BFF)** permitió separar el acceso a los servicios bancarios según las necesidades de cada canal. Se desarrollaron BFF independientes para **Web, Mobile y ATM**, cada uno con endpoints y respuestas adaptadas a su propósito.

Las pruebas realizadas permitieron comprobar la comunicación entre los BFF y el backend central, además de validar la autenticación y las principales operaciones de cada canal. Con esta solución se logra una arquitectura más organizada, segura y flexible, evitando que los clientes accedan directamente a la base de datos.

Repositorio GitHub

Repositorio del proyecto:

https://github.com/Egor-II/Exp2_S4_Backend_III.git



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.